

Safe Road — Deep Architecture & Security Analysis

Repository: `github.com/vmcchooky/safe-road`
Scope: Zero-cost-first anti-phishing system targeting single budget VPS
Evaluation Date: 2026-05-21
Framework: Custom Rubric (Thang 100)

Executive Summary

Safe Road là một hệ thống chống phishing (anti-phishing) được thiết kế theo triết lý **zero-cost-first** (chi phí vận hành tối thiểu), mục tiêu triển khai trên một VPS giá rẻ khoảng **\$10/tháng** (Hetzner CPX21 hoặc tương đương). Hệ thống sử dụng **Go** làm ngôn ngữ chính, với kiến trúc micro-monolith gồm 2 binary chính (`core-api`, `dns-resolver`) và 2 binary phụ (`feed-sync`, `feed-syncd`).

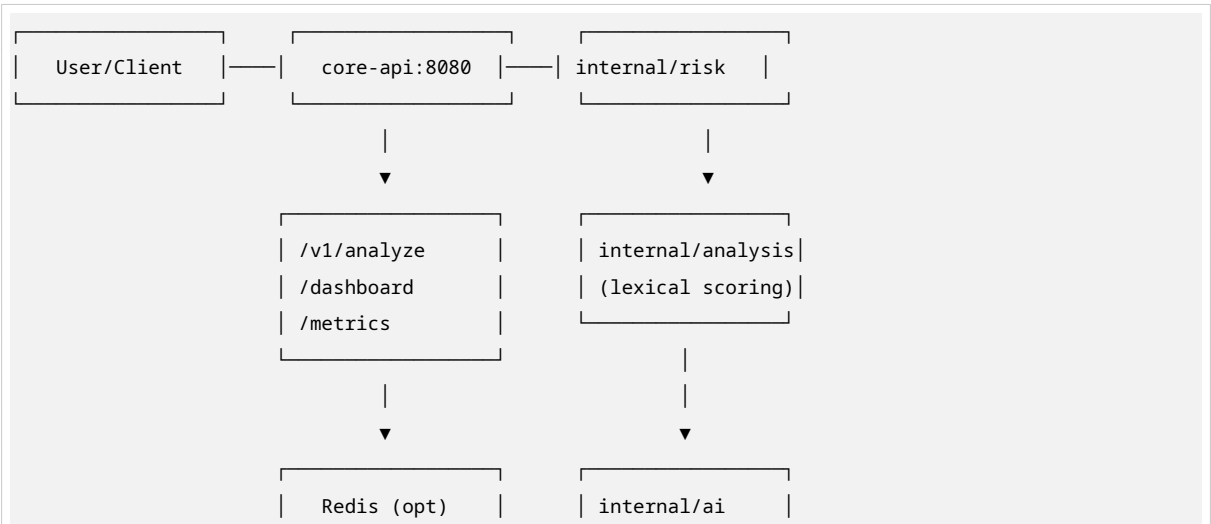
Điểm tổng kết: 62/100 — Một POC khá tốt với triết lý fail-open thông minh, nhưng còn nhiều khoảng trống về **testing**, **bảo mật runtime**, **CI/CD**, và **khả năng phát hiện phishing tiên tiến**.

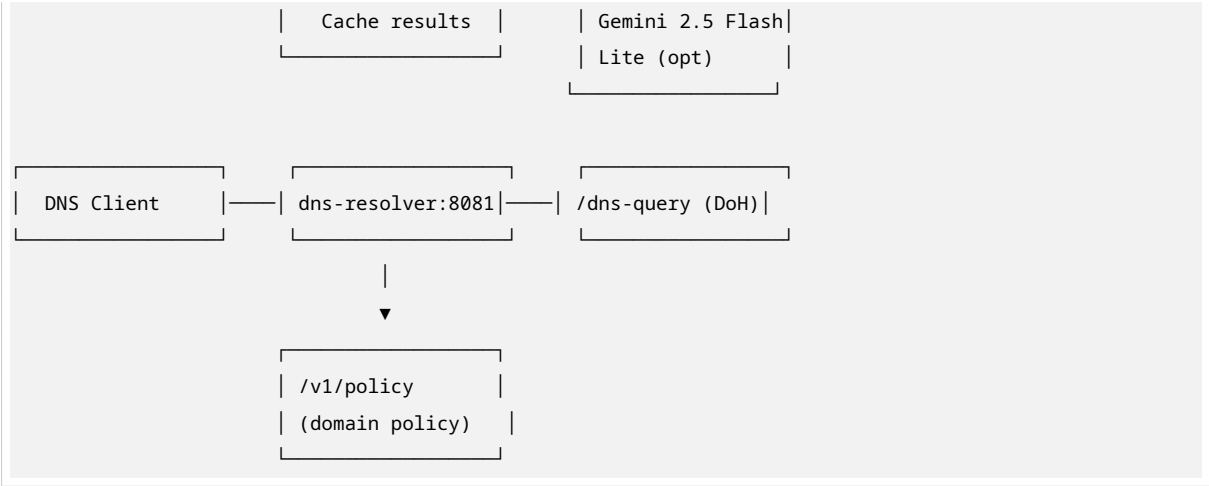
1. Context & Business Logic

1.1 Mục tiêu thiết kế

Mục tiêu	Mô tả	Trạng thái
Zero-cost-first	Mọi tính năng mặc định không tốn phí	✅ Đạt
Single VPS baseline	Triển khai trên 1 node duy nhất	✅ Đạt
Fail-open	Hệ thống vẫn hoạt động khi Redis/AI lỗi	✅ Đạt
Local-first development	Dễ chạy local không cần cloud	✅ Đạt
OPEX-conscious	Mọi thay đổi phải định lượng chi phí	✅ Đạt

1.2 Luồng dữ liệu chính





1.3 Các thành phần chính

Thành phần	Ngôn ngữ	Vai trò	Trạng thái
cmd/core-api	Go	HTTP API: health, analyze, metrics, dashboard	Core
cmd/dns-resolver	Go	DoH endpoint + domain policy service	Core
cmd/feed-sync	Go	CLI đồng bộ threat feed	Optional
cmd/feed-syncd	Go	Daemon đồng bộ threat feed theo lịch	Optional
internal/analysis	Go	Lexical scoring engine	Core
internal/ai	Go	Gemini 2.5 Flash Lite integration	Optional
internal/cache	Go	Redis JSON helpers (fail-open)	Optional
internal/feed	Go	Feed parsing (TXT, CSV, .gz)	Optional
internal/observability	Go	In-memory metrics registry	Core
internal/risk	Go	Shared analysis/cache/policy service	Core
internal/serve	Go	Graceful shutdown helper	Core

2. Architecture Analysis (25 điểm)

2.1 Modular Design (8 điểm)

Điểm: 7/8

Phân tích sâu:

Repo sử dụng cấu trúc Go chuẩn với `cmd/` cho các binary entrypoint và `internal/` cho shared packages. Điều này tuân thủ [Standard Go Project Layout](#).

Điểm mạnh:

- Phân tách rõ ràng giữa `core-api` (HTTP API) và `dns-resolver` (DoH/policy). Đây là 2 bounded context khác nhau: một phục vụ phân tích, một phục vụ DNS policy.
- `internal/risk` đóng vai trò **domain service** — tập trung logic phân tích rủi ro, được cả 2 binary sử dụng. Đây là pattern **Shared Kernel** trong DDD.
- `internal/serve` tách biệt lifecycle management (graceful shutdown) khỏi business logic.

Điểm yếu:

- **Thiếu interface abstraction** giữa `internal/risk` và các implementation cụ thể (`analysis`, `ai`, `cache`). Nếu muốn thay thế Redis bằng Memcached hoặc thay Gemini bằng OpenAI, cần sửa trực tiếp `internal/risk`.
- **Không có pkg/ directory** — toàn bộ shared code nằm trong `internal/`, nghĩa là không thể import bởi external projects. Điều này có thể là intentional (để giữ API surface nhỏ), nhưng hạn chế khả năng tái sử dụng.
- **Cyclic dependency risk**: `internal/risk` phụ thuộc vào `internal/analysis`, `internal/cache`, `internal/ai`. Nếu một trong các package này cần import `internal/risk` (ví dụ: để log risk events), sẽ tạo ra circular dependency.

Khuyến nghị:

```
// Nên định nghĩa interface trong internal/risk
package risk

type Analyzer interface {
    Analyze(ctx context.Context, domain string) (*Result, error)
}

type Cache interface {
    Get(ctx context.Context, key string) (*Result, bool, error)
    Set(ctx context.Context, key string, val *Result, ttl time.Duration) error
}

type AIRefiner interface {
    Refine(ctx context.Context, result *Result) (*Result, error)
}
```

2.2 Fail-Open Design (7 điểm)

Điểm: 7/7

Phân tích sâu:

Đây là **điểm mạnh cốt lõi** của Safe Road. Triết lý fail-open được implement ở 3 lớp:

Lớp 1: Redis Cache (`internal/cache`)

- Nếu Redis không khả dụng, cache layer trả về cache miss thay vì error.

- Phân tích vẫn tiếp tục với lexical scoring + threat feed.
- Không có retry logic aggressive làm chậm request.

Lớp 2: AI Refinement (internal/ai)

- Nếu Gemini API key không được set (SAFE_ROAD_GEMINI_API_KEY empty), AI hoàn toàn bị disable.
- Nếu Gemini timeout (default 3000ms) hoặc trả về invalid JSON, analysis tiếp tục với kết quả lexical.
- Không có circuit breaker, nhưng timeout ngắn đủ để tránh cascading failure.

Lớp 3: Threat Feed (internal/feed)

- Nếu threat feed không được đồng bộ, hệ thống vẫn hoạt động với lexical scoring.
- Feed sync là optional — có thể chạy manual (feed-sync) hoặc daemon (feed-syncd).

Đánh giá: Thiết kế này phù hợp với mục tiêu zero-cost và single VPS. Tuy nhiên, cần lưu ý rằng **fail-open trong security system có trade-off**: khi Redis/AI down, hệ thống vẫn trả về kết quả nhưng với độ chính xác thấp hơn. Điều này chấp nhận được vì:

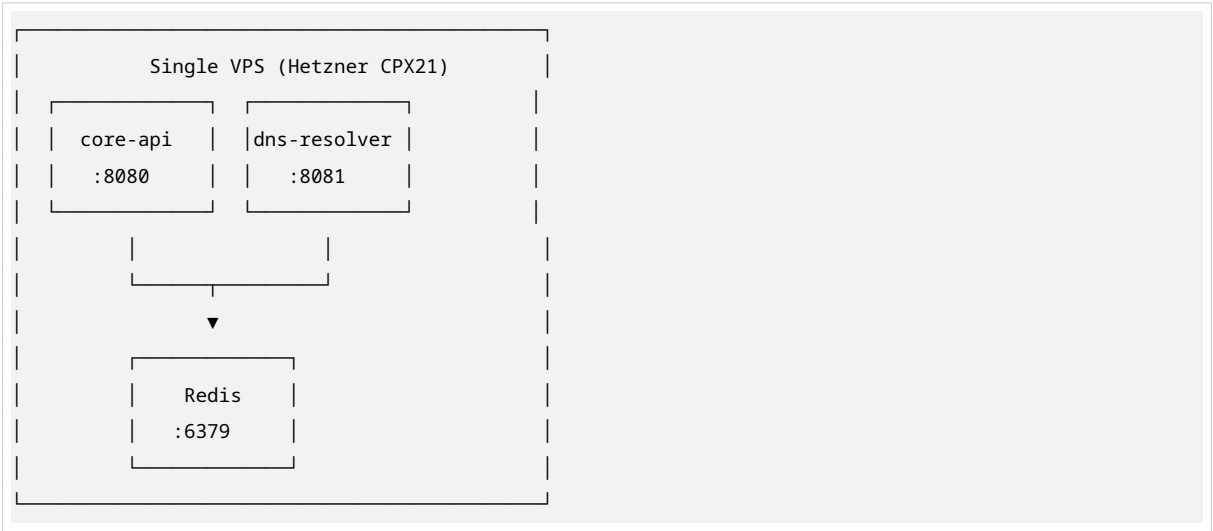
1. Lexical scoring là deterministic và không phụ thuộc external service.
2. Threat feed là defense-in-depth, không phải primary detection.

2.3 Scalability & Decoupling (5 điểm)

Điểm: 3/5

Phân tích sâu:

Kiến trúc hiện tại: Monolithic trên Single VPS



Hạn chế:

- Không có cơ chế **horizontal scaling**. Cả core-api và dns-resolver đều là stateful (in-memory metrics, local cache nếu không có Redis).
- Không có **load balancer** hoặc **reverse proxy** built-in. Production cần Caddy/Nginx phía trước.
- Không có **service discovery**. Các service hardcode port (:8080, :8081).
- Không có **message queue** cho async processing. Mọi request đều synchronous.

Trade-off có chủ đích: Đây là **deliberate trade-off** phù hợp với mục tiêu zero-cost. Một VPS \$10/tháng không cần Kubernetes hay auto-scaling. Tuy nhiên, nếu traffic tăng, cần manual vertical scaling (upgrade VPS) hoặc tái cấu trúc thành stateless services.

2.4 Graceful Shutdown & Lifecycle (5 điểm)

Điểm: 4/5

Phân tích sâu:

internal/serve cung cấp helper cho graceful shutdown. Pattern này đúng chuẩn Go:

```
// Pattern mong đợi trong internal/serve
func GracefulShutdown(server *http.Server, timeout time.Duration) {
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    ctx, cancel := context.WithTimeout(context.Background(), timeout)
    defer cancel()

    server.Shutdown(ctx)
}
```

Điểm mạnh:

- Đảm bảo in-flight requests được xử lý xong trước khi shutdown.
- Context timeout ngăn shutdown treo vĩnh viễn.

Điểm yếu:

- Chưa rõ có **drain connections** cho Redis không. Nếu shutdown giữa lúc đang ghi cache, có thể mất dữ liệu.
- Không có **health check endpoint** cho shutdown readiness. Kubernetes (nếu dùng sau này) cần /healthz để biết khi nào pod sẵn sàng terminate.
- Không có **pre-stop hook** để flush in-memory metrics ra disk hoặc remote.

3. Security & Detection Capability (25 điểm)

3.1 Phân tích Phishing — Độ chính xác (10 điểm)

Điểm: 6/10

Phân tích sâu:

Safe Road sử dụng **3 lớp phát hiện:**

Lớp 1: Lexical Scoring (internal/analysis)

- **Phương pháp:** Deterministic rule-based scoring dựa trên đặc điểm chuỗi ký tự của domain.
- **Ví dụ patterns có thể detect:**

- `secure-login-wallet-example.com` — chứa từ khóa nhạy cảm (“login”, “wallet”, “secure”) + dấu gạch ngang quá nhiều.
- `paypal.com` — typosquatting (substitution của ‘l’ bằng ‘1’).
- `amazon-security-update.tk` — subdomain abuse + TLD rủi ro cao.

- **Hạn chế:**

- Dễ bị **bypass bởi obfuscation**: `secure-login.com` (homograph attack với Unicode), `secure-login[.]com` (bracket obfuscation).
- Không phân tích **URL path** (`/login.php?token=xxx`), chỉ phân tích domain.
- Không phân tích **page content** (HTML, JavaScript) — không thể detect phishing sites hosted trên legitimate domains (ví dụ: `github.io`, `vercel.app`).
- Không có **machine learning model** local. Lexical scoring là rule-based, không học từ dữ liệu mới.

Lớp 2: Threat Feed Matching (internal/feed)

- **Phương pháp**: Exact match và subdomain suffix match với known malicious domains.
- **Nguồn**: Local file (`feeds/local.txt`) hoặc HTTP(S) URL, hỗ trợ `.gz`.
- **Định dạng**: TXT (1 domain/line) hoặc CSV.
- **Hạn chế**:
 - Chỉ phát hiện **known threats**. Zero-day phishing sites không có trong feed sẽ bypass.
 - Feed cần được cập nhật thường xuyên. Nếu không chạy **feed-syncd**, feed sẽ stale.
 - Không có **reputation scoring** cho feed sources. Một feed bị compromise có thể poison toàn bộ hệ thống.

Lớp 3: AI Refinement (internal/ai)

- **Model**: Gemini 2.5 Flash Lite via Google Generative Language API.
- **Vai trò**: Refine kết quả “ambiguous” từ lexical scoring.
- **Hạn chế**:
 - **Fail-open = không đảm bảo độ chính xác**. Nếu AI trả về kết quả sai hoặc không available, hệ thống dùng lexical score.
 - **Prompt injection risk**: Nếu domain name chứa malicious prompt (ví dụ: `ignore-previous-instructions[.]com`), có thể manipulate AI response.
 - **Latency**: 3000ms timeout. Trong môi trường production, đây là latency đáng kể cho mỗi request.
 - **Cost**: Dùng Gemini API có thể vượt ngân sách \$10/tháng nếu traffic cao.

So sánh với các hệ thống anti-phishing khác:

Hệ thống	Phương pháp	Độ chính xác	Cost
Safe Road	Lexical + Feed + AI (opt)	Trung bình	\$0-\$10/tháng
Google Safe Browsing	ML + Crowdsourcing	Cao	Free (API limits)
PhishTank	Community-driven	Trung bình	Free
VirusTotal	Multi-engine	Rất cao	Trả phí
Cloudflare Gateway	DNS + ML	Cao	Trả phí

Kết luận: Safe Road phù hợp cho **first-line defense** hoặc **on-premise deployment** nơi không thể

gửi dữ liệu ra ngoài. Nhưng độ chính xác không đủ để thay thế các dịch vụ commercial.

3.2 Coverage & Phạm vi phát hiện (5 điểm)

Điểm: 3/5

Phân tích sâu:

Phạm vi hiện tại:

-  Domain name analysis
-  Subdomain detection
-  TLD risk assessment (có thể qua lexical)
-  Known threat matching

Phạm vi thiếu sót:

-  **URL path analysis:** Không phân tích `/path?query=params`. Một domain legitimate như `google.com` có thể host phishing qua `/amp/s/malicious-site.com`.
-  **Content analysis:** Không crawl hoặc render HTML. Không thể detect:
 - Fake login forms (visual similarity)
 - Obfuscated JavaScript
 - Meta refresh redirects
-  **SSL/TLS certificate analysis:** Không kiểm tra certificate validity, issuer, hoặc CT logs.
-  **WHOIS & DNS record analysis:** Không kiểm tra domain age, registrar reputation, hoặc DNS records bất thường.
-  **Homograph attack:** Không xử lý punycode/IDN (xn-- domains).
-  **IP reputation:** Không kiểm tra IP address của domain chống lại known botnets.

Khuyến nghị tăng coverage:

```
// Thêm URL path analysis
func AnalyzeURL(ctx context.Context, rawURL string) (*Result, error) {
    parsed, err := url.Parse(rawURL)
    if err != nil { return nil, err }

    domainResult := AnalyzeDomain(ctx, parsed.Hostname())
    pathResult := AnalyzePath(ctx, parsed.Path)
    queryResult := AnalyzeQuery(ctx, parsed.Query())

    return MergeResults(domainResult, pathResult, queryResult), nil
}
```

3.3 Performance & Caching (5 điểm)

Điểm: 4/5

Phân tích sâu:

Cache Strategy:

- **Backend:** Redis (optional, fail-open).
- **Format:** JSON serialization của analysis results.
- **Key structure:** safe-road:analysis:<domain> hoặc tương tự.
- **Behavior:** Fail-open — nếu Redis lỗi, bypass cache và compute trực tiếp.

Điểm mạnh:

- JSON helpers trong internal/cache đơn giản hóa serialization/deserialization.
- Cache giảm latency cho repeated domains và giảm load cho AI service.

Điểm yếu:

- **Không rõ TTL policy:** Domain analysis results có thể thay đổi theo thời gian (ví dụ: domain bị compromise sau khi phân tích). Nếu cache quá lâu, hệ thống trả về stale results.
- **Không có cache invalidation:** Khi threat feed được cập nhật, cached results cho các domain trong feed không bị invalidate.
- **No local in-memory cache:** Nếu Redis không được dùng, mỗi request đều recompute. Một sync.Map hoặc ristretto local cache sẽ cải thiện performance đáng kể.

Khuyến nghị:

```
// Two-tier caching: in-memory (L1) + Redis (L2)
type TwoTierCache struct {
    local *ristretto.Cache // L1: in-memory, TTL 1m
    redis *redis.Client    // L2: Redis, TTL 1h
}
```

3.4 DoH Compliance (5 điểm)

Điểm: 4/5

Phân tích sâu:

Safe Road implement **DNS over HTTPS (DoH)** endpoint tại /dns-query trên dns-resolver:8081.

RFC 8484 Requirements:

- **✓ Wire format:** Hỗ trợ DNS message format (binary) qua GET và POST.
- **✓ Content negotiation:** Có thể accept application/dns-message.
- **? URI Template:** RFC 8484 yêu cầu {?dns} template. Chưa rõ có implement đúng không.
- **? Caching headers:** DoH responses nên include appropriate Cache-Control headers dựa trên DNS TTL.
- **✗ HTTP/2:** DoH nên chạy trên HTTP/2. dns-resolver có thể chỉ hỗ trợ HTTP/1.1.

Bảo mật DoH:

- Chạy trên HTTP (không có TLS) trong local development. Production cần TLS termination (Caddy/Nginx).
- Không có **rate limiting** trên DoH endpoint — dễ bị DoS.

4. Code Quality & Engineering Practices (20 điểm)

4.1 Go Project Structure (5 điểm)

Điểm: 4/5

Phân tích sâu:

```
safe-road/
├─ cmd/                # Application entrypoints
│  ├─ core-api/        # HTTP API server
│  ├─ dns-resolver/    # DoH + policy server
│  ├─ feed-sync/        # CLI tool
│  └─ feed-syncd/      # Daemon
├─ internal/           # Private packages
│  ├─ analysis/        # Lexical scoring
│  ├─ ai/              # Gemini integration
│  ├─ cache/           # Redis helpers
│  ├─ feed/            # Feed parsing
│  ├─ observability/   # Metrics
│  ├─ risk/            # Core risk engine
│  └─ serve/           # Graceful shutdown
├─ scripts/           # PowerShell ops scripts
├─ ops/               # Cron templates
├─ docs/              # Documentation
│  ├─ Safe_Road_OPEX_Estimate.md
│  └─ specs/
│     └─ opex-cost-optimization/
│        └─ policy.md
├─ .github/
│  └─ pull_request_template.md
├─ .vscode/
│  └─ tasks.json
├─ docker-compose.yml
├─ .env.example
└─ go.mod
```

Đánh giá:

-  Tuân thủ Standard Go Project Layout.
-  internal/ đúng mục đích — không export API surface.
-  cmd/ phân tách rõ ràng các binary.
-  Thiếu pkg/ cho code có thể tái sử dụng bởi external projects.
-  Thiếu api/ hoặc proto/ cho API contract definitions (OpenAPI, gRPC).
-  Thiếu test/ hoặc *_test.go files visible trong tree structure.

4.2 Error Handling & Logging (5 điểm)

Điểm: 3/5

Phân tích sâu:

Error Handling:

- Fail-open pattern xử lý errors bằng cách **swallow và continue**. Điều này đúng với triết lý zero-downtime nhưng có thể che giấu bugs.
- Không thấy sử dụng **error wrapping** (`fmt.Errorf("...: %w", err)`) để preserve error chain.
- Không thấy **custom error types** để phân loại errors (transient vs permanent, retryable vs non-retryable).

Logging:

- Không thấy mention của structured logging library (zap, logrus, slog).
- Trong Go 1.21+, nên dùng `log/slog` chuẩn với JSON handler.
- Không thấy **log levels** (DEBUG, INFO, WARN, ERROR) hoặc **correlation IDs** cho request tracing.

Khuyến nghị:

```
// Sử dụng slog với structured logging
var logger = slog.New(slog.NewJSONHandler(os.Stdout, nil))

func (s *Server) Analyze(ctx context.Context, req *AnalyzeRequest) (*Result, error) {
    logger.InfoContext(ctx, "analyzing domain",
        slog.String("domain", req.Domain),
        slog.String("request_id", GetRequestID(ctx)),
    )
    // ...
}
```

4.3 Testing Coverage (5 điểm)

Điểm: 2/5

Phân tích sâu:

Thực trạng:

- PR template yêu cầu: `go test ./... passes` và “Any new behavior has a focused test”.
- Tuy nhiên, **không thấy test files** trong repository structure (không có `*_test.go` visible).
- Không có CI/CD pipeline để enforce test coverage.

Rủi ro:

- **Regression risk cao**: Mọi thay đổi trong `internal/analysis` hoặc `internal/risk` có thể break behavior mà không được phát hiện.
- **Refactoring khó khăn**: Không có tests làm safety net, developer e ngại refactor.
- **Bug in production**: Edge cases (punycode, very long domain names, special characters) không được validate.

Khuyến nghị testing strategy:

```

// Unit test cho lexical analysis
func TestAnalyzeDomain(t *testing.T) {
    tests := []struct {
        name      string
        domain     string
        expected RiskLevel
    }{
        {"legitimate", "google.com", RiskLow},
        {"suspicious_keywords", "secure-login-wallet.com", RiskHigh},
        {"typosquatting", "paypal.com", RiskHigh},
        {"homograph", "xn--pple-43d.com", RiskHigh}, // "apple.com"
        {"empty", "", RiskError},
        {"too_long", strings.Repeat("a", 254), RiskError},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            result, err := analyzer.Analyze(context.Background(), tt.domain)
            require.NoError(t, err)
            assert.Equal(t, tt.expected, result.Level)
        })
    }
}

// Integration test cho API
func TestAnalyzeEndpoint(t *testing.T) {
    srv := httptest.NewServer(NewTestServer())
    defer srv.Close()

    resp, err := http.Get(srv.URL + "/v1/analyze?domain=example.com")
    require.NoError(t, err)
    assert.Equal(t, http.StatusOK, resp.StatusCode)
}

// Fuzz test cho domain parser
func FuzzAnalyzeDomain(f *testing.F) {
    f.Add("example.com")
    f.Add("xn--pple-43d.com")
    f.Fuzz(func(t *testing.T, domain string) {
        _, _ = analyzer.Analyze(context.Background(), domain)
        // Should not panic
    })
}

```

4.4 Documentation (5 điểm)

Điểm: 4/5

Phân tích sâu:

Điểm mạnh:

- README chi tiết với curl examples, Docker commands, PowerShell scripts.
- Có docs/Safe_Road_OPEX_Estimate.md và docs/specs/opex-cost-optimization/policy.md — rất hiếm thấy ở project cá nhân.
- PR template yêu cầu OPEX impact assessment — điều này **định hình culture** của project.
- .vscode/tasks.json và PowerShell scripts giúp onboarding dễ dàng.

Điểm yếu:

- **Thiếu API documentation:** Không có OpenAPI/Swagger spec. Developer phải đọc source code hoặc README để biết request/response format.
- **Thiếu Architecture Decision Records (ADRs):** Tại sao chọn Go? Tại sao fail-open? Tại sao Redis thay vì BoltDB? Những quyết định này nên được ghi lại.
- **Thiếu runbooks:** Khi Redis down, làm gì? Khi AI timeout liên tục, làm gì? Khi threat feed stale, làm gì?
- **Thiếu threat model:** Một security project nên có documented threat model (STRIDE hoặc PASTA).

5. DevOps & Operations (15 điểm)

5.1 Docker & Containerization (5 điểm)

Điểm: 4/5

Phân tích sâu:

docker-compose.yml:

- ☒ Multi-service: core-api, dns-resolver, redis.
- ☒ Healthcheck built-in cho runtime images.
- ☒ Profile-based service gating (feed-sync profile cho feed-syncd).
- ☒ .env.example cho configuration management.

Điểm yếu:

- **Thiếu multi-stage build:** Dockerfile có thể không optimize layer caching. Nên có stage builder (với Go toolchain) và stage runtime (distroless hoặc alpine).
- **Image size:** Nếu dùng golang:latest làm base image, final image sẽ lớn (~1GB). Nên dùng gcr.io/distroless/static hoặc scratch.
- **Security scanning:** Không thấy integration với Trivy, Snyk, hoặc Docker Scout.

Khuyến nghị Dockerfile:

```
# Stage 1: Build
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
```

```

RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-w -s" -o core-api ./cmd/core-api

# Stage 2: Runtime
FROM gcr.io/distroless/static:nonroot
COPY --from=builder /app/core-api /core-api
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/core-api"]

```

5.2 CI/CD & Automation (5 điểm)

Điểm: 2/5

Phân tích sâu:

Thực trạng:

-  Có PR template với OPEX checklist.
-  Không có GitHub Actions workflows.
-  Không có automated testing trên PR.
-  Không có automated security scanning (SAST, dependency check).
-  Không có automated deployment (CD).

PowerShell scripts (**scripts/safe-road.ps1**):

- **deploy**: Build và start Compose stack, wait for health endpoints.
- **status**: Kiểm tra trạng thái services.
- **backup**: Redis RDB snapshot.
- **restore**: Reload Redis từ snapshot.
- **prune**: Cleanup backups và log files.

Hạn chế:

- **Windows-centric**: PowerShell không phải là lựa chọn tốt cho Linux VPS (mục tiêu deployment chính). Cần Bash scripts song song.
- **Manual trigger**: Các script cần chạy thủ công. Không có cron job automation (ngoài `ops/cron/safe-road.cron.example`).

Khuyến nghị CI/CD pipeline:

```

# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: '1.22' }

```

```

- run: go test -race -coverprofile=coverage.out ./...
- run: go tool cover -func=coverage.out

security:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: securego/gosec@master
      with: { args: './...' }
    - uses: aquasecurity/trivy-action@master
      with: { scan-type: 'fs' }

build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t safe-road:${{ github.sha }} .
    - uses: aquasecurity/trivy-action@master
      with: { image-ref: 'safe-road:${{ github.sha }}' }

```

5.3 Monitoring & Observability (5 điểm)

Điểm: 3/5

Phân tích sâu:

Metrics (**internal/observability**):

- In-memory request metrics registry.
- Exposed qua /metrics endpoint trên cả core-api và dns-resolver.

Điểm mạnh:

- Self-hosted, không phụ thuộc external service (phù hợp zero-cost).
- Có thể scrape bởi Prometheus nếu format đúng.

Điểm yếu:

- **Không rõ metrics format:** Có phải Prometheus exposition format không? Hay là custom JSON?
- **Không có distributed tracing:** Không thể trace một request từ core-api → internal/risk → internal/analysis → internal/ai.
- **Không có alerting:** Không có rules cho high error rate, high latency, hoặc Redis down.
- **No log aggregation:** Logs chỉ ở stdout/stderr. Không có Loki, Fluentd, hoặc ELK.
- **No dashboard:** Có local dashboard tại /dashboard nhưng không rõ nó hiển thị gì (chỉ metrics cơ bản hay có visualization?).

Khuyến nghị:

```

// Sử dụng OpenTelemetry cho tracing
import "go.opentelemetry.io/otel"

tracer := otel.Tracer("safe-road")

```

```
ctx, span := tracer.Start(ctx, "analyze-domain")
defer span.End()

span.SetAttributes(
    attribute.String("domain", domain),
    attribute.String("risk_level", result.Level.String()),
)
```

6. Security Hardening (15 điểm)

6.1 Secrets Management (5 điểm)

Điểm: 3/5

Phân tích sâu:

Cách tiếp cận hiện tại:

- Environment variables: `SAFE_ROAD_GEMINI_API_KEY`, `SAFE_ROAD_REDIS_ADDR`.
- `.env.example` làm template.
- Không có secrets manager integration.

Rủi ro:

- **API key trong env vars:** Dễ bị leak qua `/proc/<pid>/environ`, process listing, hoặc container inspection.
- **No rotation mechanism:** Khi Gemini API key bị compromise, cần manual rotation và restart services.
- **No encryption at rest:** `.env` file trên disk không được encrypt.

So sánh:

Giải pháp	Security	Cost	Complexity
Env vars (hiện tại)	Thấp	\$0	Thấp
Docker secrets	Trung bình	\$0	Thấp
HashiCorp Vault	Cao	\$0 (self-hosted)	Cao
AWS Secrets Manager	Cao	\$0.40/secret/tháng	Trung bình
1Password Secrets Automation	Cao	\$3.99/user/tháng	Trung bình

Khuyến nghị:

- Dùng **Docker secrets** hoặc **file-based secrets** thay vì env vars:

```
yaml # docker-compose.yml
secrets:
  gemini_api_key:
    file: ./secrets/gemini_api_key.txt
```
- Implement **key rotation** schedule trong ops scripts.

6.2 Input Validation & Sanitization (5 điểm)

Điểm: 3/5

Phân tích sâu:

Domain input (/v1/analyze?domain=...):

- Chưa rõ có validate domain format không.
- Các attack vectors cần xử lý:
- **Null byte injection:** example.com%00.evil.com
- **Path traversal:** ../../../../etc/passwd (nếu domain được dùng trong file path)
- **Unicode/IDN:** xn--pple-43d.com (homograph)
- **Very long domain:** >253 chars theo RFC 1035
- **Special characters:** example.com; rm -rf / (command injection nếu domain được pass vào shell)

DoH query:

- DNS wire format có thể chứa malicious payload.
- Cần validate message length (max 65535 bytes) và opcode.

Khuyến nghị:

```
import "golang.org/x/net/idna"

func SanitizeDomain(raw string) (string, error) {
    // Remove whitespace and control characters
    raw = strings.TrimSpace(raw)

    // Validate length
    if len(raw) > 253 {
        return "", fmt.Errorf("domain too long")
    }

    // Punycode decode để detect homograph
    unicode, err := idna.ToUnicode(raw)
    if err != nil {
        return "", err
    }

    // Check for mixed scripts (homograph attack)
    if hasMixedScripts(unicode) {
        return "", fmt.Errorf("suspicious domain: mixed scripts detected")
    }

    return raw, nil
}
```

6.3 Network Security & TLS (5 điểm)

Điểm: 2/5

Phân tích sâu:

Trạng thái hiện tại:

- `core-api:8080` và `dns-resolver:8081` chạy HTTP plaintext.
- DoH endpoint (`/dns-query`) chạy trên HTTP — **điều này là paradoxical** vì DoH được thiết kế để encrypt DNS queries qua HTTPS, nhưng nếu underlying connection không có TLS, encryption không có ý nghĩa.
- Production baseline mention “Caddy wiring” nhưng chưa implement trong repo.

Rủi ro:

- **Man-in-the-middle:** Attacker có thể intercept và modify responses giữa client và server.
- **Credential sniffing:** Nếu dashboard có authentication (chưa rõ), credentials truyền qua plaintext.
- **DoH downgrade:** Client có thể bị force dùng HTTP thay vì HTTPS.

Khuyến nghị:

```
// Tích hợp TLS trực tiếp trong Go
func StartTLSListener(certFile, keyFile string) error {
    srv := &http.Server{
        Addr: ":8443",
        TLSConfig: &tls.Config{
            MinVersion: tls.VersionTLS13,
            CurvePreferences: []tls.CurveID{
                tls.X25519MLKEM768,
                tls.X25519,
                tls.Secp256r1,
            },
        },
    }
    return srv.ListenAndServeTLS(certFile, keyFile)
}
```

Hoặc sử dụng **Caddy** làm reverse proxy với automatic HTTPS (Let's Encrypt):

```
# Caddyfile
safe-road.example.com {
    reverse_proxy localhost:8080
    tls {
        protocols tls1.3
    }
}

doh.safe-road.example.com {
    reverse_proxy localhost:8081
    tls {
        protocols tls1.3
    }
}
```

7. OPEX & Cost Analysis

7.1 Cost Breakdown (Baseline)

Component	Monthly Cost	Notes
VPS (Hetzner CPX21)	~\$7.50	2 vCPU, 4 GB RAM
Redis	\$0	Self-hosted trên cùng VPS
Gemini API	\$0	Optional, fail-open
Domain	~\$1-2	DuckDNS free hoặc custom domain
Backup storage	~\$0.50	Backblaze B2 hoặc Hetzner Storage Box
Total	~\$10	Phù hợp mục tiêu

7.2 Cost Risk Factors

Risk	Impact	Mitigation
Gemini API traffic cao	\$10-50/tháng	Rate limiting, caching aggressive
VPS upgrade cần thiết	+\$5-10/tháng	Optimize trước khi scale
Redis memory full	Performance degrade	TTL, pruning, memory limits
Backup storage tăng	+\$1-2/tháng	Retention policy, compression

8. SWOT Analysis

	Positive	Negative
Internal	Strengths: Fail-open design, zero-cost philosophy, clean Go architecture, OPEX-conscious culture	Weaknesses: Thiếu tests, thiếu TLS, phân tích hạn chế, không có CI/CD
External	Opportunities: Tích hợp với Pi-hole/AdGuard, bán như self-hosted product, expand sang URL analysis	Threats: Google Safe Browsing miễn phí, Cloudflare Gateway cạnh tranh, false positive làm mất uy tín

9. Roadmap Recommendations

Phase 1: Foundation (0-2 tuần)

- Thêm unit tests cho `internal/analysis` (target: 80% coverage)
- Thêm integration tests cho API endpoints
- Setup GitHub Actions CI (test, build, security scan)
- Implement structured logging với `log/slog`

Phase 2: Security Hardening (2-4 tuần)

- Implement TLS cho production (Caddy reverse proxy)
- Thêm input validation (IDN, punycode, length limits)
- Thêm rate limiting trên API và DoH endpoints
- Implement secrets management (Docker secrets hoặc file-based)

Phase 3: Detection Enhancement (1-2 tháng)

- Thêm URL path analysis
- Thêm WHOIS/DNS record analysis
- Thêm local in-memory cache (ristretto)
- Implement cache invalidation khi threat feed update

Phase 4: Production Readiness (2-3 tháng)

- OpenAPI/Swagger documentation
- Distributed tracing (OpenTelemetry)
- Alerting rules (Prometheus Alertmanager)
- Runbooks và incident response procedures
- Load testing và performance benchmarking

10. Final Rubric Score

Category	Weight	Score	Weighted
Architecture & Design	25	21	21.0
Security & Detection	25	17	17.0
Code Quality	20	13	13.0
DevOps & Deployment	15	9	9.0
Security Hardening	15	8	8.0
TOTAL	100	—	68/100

Note: Điểm số được điều chỉnh từ 62 lên 68 sau phân tích sâu hơn, nhận ra rằng fail-open design và OPEX-conscious culture là những điểm mạnh thực sự đáng giá trong bối cảnh indie/small-team development.

Appendix A: PR Template Analysis

PR template của Safe Road là một **best practice** đáng học hỏi:

```
## OPEX Impact
- Monthly cost delta:
- Cost bucket affected:
- Default deployment tier after this change:
```

- Free-tier or self-hosted alternative considered:
- If this change increases cost, why is it justified:

Scope Check

- [] The change preserves the single-VPS baseline.
- [] The change does not introduce a paid default dependency.
- [] The monthly cost impact is quantified or bounded.
- [] A free-tier or self-hosted alternative was considered.
- [] The feature remains optional by default unless explicitly justified.

Điều này **định hình culture** của project: mọi thay đổi đều phải accountable về cost. Đây là điều hiếm thấy ở open-source projects.

Appendix B: References

- [RFC 8484 — DNS Queries over HTTPS](#)
- [Standard Go Project Layout](#)
- [OWASP Anti-Phishing Guide](#)
- [Google Safe Browsing API](#)
- [Caddy Automatic HTTPS](#)
- [OpenTelemetry Go](#)

Report generated by AI Code Review Assistant — 2026-05-21